Axona: A Learning-Adaptive Distributed Hash Table and Axonal Publish/Subscribe

v0.3.53 2026-05-18

David A. Smith
Axona.net
davidasmith@gmail.com

Abstract—Distributed hash tables (DHTs) are the substrate of content-addressable peer-to-peer systems, but the canonical Kademlia design routes messages in $O(\log N)$ hops without regard to network locality, yielding multi-second tail latency on million-node networks. We present Axona, a learning-adaptive DHT and axonal publish/subscribe protocol, and report two technical contributions and a deployed implementation. First, the Geographic DHT (G-DHT) embeds an S2 cell prefix in node identifiers so that XOR distance in ID space approximates physical distance — a simple structural change that reduces 500 km regional lookup latency by 3.4× over Kademlia at 25 000 nodes. Second, Axona introduces vitality — a per-edge score weight × recency in which the weight is reinforced by successful traffic (Hebbian long-term potentiation) and recency exponentially decays after a synaptic-tagging-style protection window. A single vitality-based admission gate replaces five distinct mechanisms in the predecessor NX-17, consolidating eighteen specialized rules and forty-four parameters into twelve each. The current Axona implementation, NH-1, embodies this consolidation. On a 50 000-node simulator, NX-17 routes at 1.18× the Dabek 3δ analytical floor [1] — within 36 ms of the analytical lower bound for any recursive $O(\log N)$ DHT — and NH-1 plateaus at 1.28×. Axona provides built-in publish/subscribe via axonal delivery trees, achieving 100% baseline and 100% recovered delivery under 5% churn, and recovers from single-bridge network partitions through cooperative re-stitching driven by hop caching and triadic closure. An ablation that strips the G-DHT prefix shows learning alone outperforms Kademlia by 26%, demonstrating that the contribution is structural rather than a consequence of locality engineering. NH-1 is the implementation currently deployed in the Axona peer-to-peer mesh (https://axona.net); when this paper refers to NH-1, it refers to the same code path validated in production browsers across real WebRTC networks. Naming convention used throughout: Axona names the protocol described here, N-DHT (Neuromorphic DHT) names the broader family of learning-adaptive DHT designs to which Axona belongs, and NH-1 names the specific implementation against which the empirical results in this paper were measured.

I. Introduction

Peer-to-peer applications — file sharing, blockchain peer discovery, content addressing, decentralized identity, censorship-resistant messaging — depend on a name resolution substrate that no single party controls. The canonical answer for two decades has been the distributed hash table [2]–[6], which assigns every key and every node an address in a shared identifier space, then routes messages from a key to its responsible node in $O(\log N)$ hops. Kademlia [3], the dominant production deployment (BitTorrent Mainline, Ethereum devp2p, IPFS, Tor v3 hidden services), uses an XOR distance metric over 160-bit identifiers and provably halves the distance to the destination on every hop.

Hop count, however, is not latency. Random identifiers spread peers homogeneously across the globe; the average pair sits roughly half the planet apart, with about 100 ms one-way round-trip time. With $N = 10^6$ nodes, $\log_2 N \approx 20$, and the naive estimate gives an average message time of $20 \times 100 = 2$ seconds. For real-time applications — pub/sub, voice, gaming, live coordination — this is unacceptable. The latency problem is the dominant practical barrier between DHT theory and DHT deployment.

a) Locality is necessary but not sufficient.: The first generation of latency-aware DHTs put network proximity in the routing layer itself. Pastry [4] maintains a three-tier routing state (a prefix-matched routing table R, a numerically-closest leaf set L, and an RTT-closest neighborhood set M), populating each table slot with the closest node by RTT among candidates with the appropriate prefix. Tapestry [5] pushes the same instinct further, storing soft-state location pointers along the publish path and recovering the closest replica via path intersection. Coral [7] relaxes the strict-DHT promise and builds nested RTT clusters explicitly. Vivaldi [8] synthesizes per-node Euclidean coordinates that predict pair-wise RTT without active probing.

These designs reduce the per-hop constant. They do not, however, learn from the traffic that flows through them. A Pastry routing table is set at insertion time and refreshed lazily. A Coral cluster hierarchy is determined by RTT thresholds, not by which paths are actually used. Vivaldi predicts latency but does not act on those predictions in routing without a separate decision layer. The frontier is a routing fabric whose composition adapts to observed traffic — where which peers are kept is a function of which paths the network actually carries.

b) The analytical floor.: Dabek et al. [1] proved that for any recursive $O(\log N)$ DHT, total lookup latency

converges to a hard analytical floor:

$$\text{lookup ms} \approx \delta + \frac{\delta}{2} + \frac{\delta}{4} + \cdots = 3\delta \tag{1}$$

where δ is the median pairwise one-way internet latency. Each successive lookup hop covers a geometrically halving fraction of the remaining ID-space, so the second-to-last hop is half the cost of the last, the third-to-last a quarter, and so on. The series converges regardless of N. Even an oracle that always picks the lowest-RTT finger cannot beat this bound. Until now, no published DHT has been measured at this floor.

c) Contributions.: This paper presents two protocol contributions and three empirical results that together establish a new operating point for the DHT design space:

1) The Geographic DHT (G-DHT). A simple structural extension of Kademlia: the top 8 bits of every node identifier encode the S2 cell of the node's claimed geographic position. With this prefix, XOR distance in identifier space approximates physical distance, and a node's bootstrap routing table is biased toward local peers without any change to the routing algorithm itself. G-DHT reduces 500 km regional lookup latency by 3.4× over Kademlia at 25 000 nodes (§VI), and is the substrate on which Axona's learning is layered.
2) Axona (current implementation NH-1), a learning-adaptive DHT layered above the Kademlia/G-DHT substrate, that consolidates the 18-rule, 44-parameter NX-17 lineage into a 12-rule, 12-parameter protocol governed by a single vitality-based admission gate (§IV). The consolidation is a direct application of Hebbian long-term potentiation [9], [10]: every successful lookup reinforces the synaptic weights along the traversed path, every unused edge decays over time, and the routing table evolves toward a traffic-shaped optimum.
3) First measurement of an $O(\log N)$ DHT at the Dabek 3δ analytical floor. NX-17 plateaus within 1.18× the floor at 25 000 and 50 000 nodes; NH-1 plateaus at 1.28×. Kademlia worsens from 2.01× to 2.65× over the same range (§VII-B).
4) Axonal publish/subscribe, a built-in delivery primitive structurally equivalent to SCRIBE [11] but adaptive under churn. Axona achieves 100% baseline delivery and 100% recovered delivery at 5% churn through routed re-subscribe alone — no replication, no gossip, no parent tracking (§V).
5) Geography ablation. Stripping the G-DHT prefix and re-running the comparison shows that with no geographic structure, NX-17 still routes 26% faster than Kademlia and Axona (NH-1) routes 8% faster (§VII-C). Learning is doing real work, not sharpening pre-existing geographic structure.

The contributions rest on a publicly available, open-source simulator (∼25 000 lines of JavaScript) whose every CSV is in the repository [12].

d) Roadmap.: §II surveys the lineage and related work. §III presents the neuromorphic substrate — the biology of long-term potentiation and its precise mapping into routing-table mechanics. §IV describes Axona's five operations, twelve rules, and vitality function in detail (as embodied in NH-1). §V covers axonal pub/sub. §VI covers the geographic prefix used as a bootstrap seed. §VII reports the four headline results. §VIII discusses limitations, including a third-party red-team analysis identifying deploy blockers [13]. §X sketches future work, and §XI concludes.

II. Background and Related Work

a) The Berkeley/Cambridge DHT lineage.: Axona inherits structurally from three foundational designs. Kademlia [3] provides the substrate: 160-bit random identifiers, a symmetric XOR distance metric (which makes routing self-organizing because every incoming or outgoing message updates a k-bucket), $K = 20$ peers per stratum, and α-parallel asynchronous find_node queries. Kademlia's empirical observation that old peers are more reliable than new peers [14] becomes Axona's vitality recency multiplier. Pastry [4] contributes the three-tier routing state (R, L, M), the locality-preserving join algorithm in which a new node X takes row n of its routing table from the n-th node along its routed-join path, and SCRIBE [11], the structural ancestor of axonal pub/sub. Tapestry [5] contributes the locality-aware routing table (closest node by RTT per slot, built at insertion), soft-state location pointers along publish paths, and surrogate routing (when the deterministic root is dead, route to the closest live identifier). Axona retains all of these mechanisms in some form; the contribution is to make them learn from observed traffic.

b) Latency analysis.: Dabek et al. [1] (NSDI 2004) prove the 3δ floor for any recursive $O(\log N)$ DHT and show DHash++ approaches it through proximity neighbor selection, server selection over erasure-coded fragments, and the integration of routing and fetching. Tapestry's companion measurement [5] reports a complementary metric — relative delay penalty (RDP) — with median wide-area RDP near 1 and 90th-percentile near 2–3. These analyses set the methodological standard for DHT-latency reporting and the target Axona aims to match.

c) Adaptive maintenance.: A recurring instinct in the post-2003 DHT literature is that routing-table maintenance should adapt to observed conditions rather than run on a fixed schedule. Mahajan, Castro, and Rowstron [15] introduce a cost-of-reliability framework. Krishnamurthy et al. [16] model Chord under Poisson churn analytically. Ghinita and Teo [17] present the closest existing match in mechanism family to NH-1: each peer estimates local node-failure rate μ, join rate λ, and network size N; computes pointer-staleness probabilities; and triggers cheap liveness checks or expensive accuracy checks only when warranted.

Their headline — 3.4× lower lookup failure at 2.4× lower communication overhead under variable churn — demonstrates that self-tuning works. Axona applies the same instinct to a different observable: not maintenance rate but routing-table composition.

d) Resilience and load.: Castro et al. [18] prove the foundational triplet for Byzantine resistance: constrained routing tables, secure node-ID assignment, and redundant routing. Harvesf and Blough [19] make the redundant-routing piece concrete with MaxDisjoint replica placement, achieving 90% lookup success at 50% malicious nodes. Makris et al. [20] address the orthogonal problem of Zipf-distributed request rates with the Directory For Exceptions mechanism — a hybrid of consistent hashing [21] and a small distributed override that migrates hot keys to less-loaded peers. Axona's §X identifies these as the canonical mitigations for two open security and load-balancing gaps.

e) Locality alternatives.: Beyond Pastry/Tapestry's structural locality, two adaptive alternatives warrant comparison. Coral [7] builds nested clusters at increasing RTT thresholds, relaxing the strict DHT promise (sloppy hashing) to deliver content-distribution at scale. Vivaldi [8] computes synthetic per-node coordinates that predict pair-wise RTT to peers a node has never directly contacted. Both treat the network as something to measure and adapt to. Axona goes further: rather than measure or predict, it learns paths themselves through reinforcement on the routing layer.

III. The Neuromorphic Substrate

The framing of Axona as a "neuromorphic" DHT (placing it in the N-DHT family) is not decorative. The brain and a peer-to-peer DHT face a precisely shared engineering problem, and the solution that biological evolution found applies to overlay routing with no metaphorical slippage.

a) The shared engineering problem.: A single neuron carries on the order of 10^4 synapses — a hard upper bound set by the energetic cost of maintaining synaptic machinery [22]. The number of candidate synaptic partners — other neurons whose axons could reach it — is orders of magnitude larger. The neuron cannot store every co-firing event; it must score, prioritize, and prune. A peer-to-peer node in a browser deployment is bounded by WebRTC's connection limit (typically 50–200 peers, with Safari at $\sim$95). The number of candidate peers in a 50 000-node overlay is three orders of magnitude larger. The same storage-constrained, more-candidates-than-capacity problem appears in both systems. The biological solution is the engineering solution.

b) Long-term potentiation.: The mechanism by which the brain remembers which paths are useful is long-term potentiation (LTP), discovered by Bliss and Lømo [10] in rabbit hippocampus. A few high-frequency electrical bursts to the perforant path produced post-synaptic responses that remained elevated for weeks. The molecular cascade that underlies LTP gives the brain a coincidence-aware, durable, and selectively reversible learning signal:

- NMDA receptors as coincidence detectors. Glutamate from the pre-synaptic neuron only opens NMDA channels if the post-synaptic neuron is already depolarized. The signal is "I fired and you fired," not just "I fired."
- Calcium-mediated AMPA insertion. Calcium influx through opened NMDA channels triggers insertion of additional AMPA receptors into the post-synaptic membrane. The synapse is now physically more sensitive to subsequent glutamate release.
- Consolidation. Within roughly 30 minutes, gene transcription and new protein synthesis produce a durable physical change. Sometimes new dendritic spines form. The synapse is re-weighted permanently [23].

The opposing process, long-term depression (LTD), weakens synapses under low-frequency stimulation [24]. LTP and LTD together let the brain learn which routes among neurons are worth keeping.

c) Synaptic tagging and capture.: Frey and Morris [25] added a critical refinement: recently potentiated synapses carry a transient "tag" that protects them from being overwritten by competing potentiation events, but only for a brief consolidation window. The tag bridges the timescale gap between the coincidence detection (milliseconds) and the protein synthesis that consolidates the change (tens of minutes). Without it, learning would be self-destructive: the very synapses just discovered to be useful would be the most likely to be overwritten by the next high-frequency event. Axona's INERTIA_DURATION parameter (NH-1) implements precisely this protection in routing.

d) The Hebbian rule.: Donald Hebb's 1949 formulation [9] predates the molecular discoveries by two decades and remains the cleanest abstraction:

> Neurons that fire together, wire together.

Every weight in every artificial neural network ultimately invokes this rule. Axona applies it not to perception but to routing: peers that successfully carry traffic together earn higher weights; peers that carry no traffic decay. The routing table is shaped by the queries it serves, in exactly the way a cortical region's synaptic structure is shaped by the sensory input it processes.

e) Mapping table.: The translation from neuroscience mechanisms to Axona source-code mechanisms (as embodied in the NH-1 implementation) is direct, not analogical (Figure 1, Table I). Every term has one corresponding artifact in the codebase.

f) Looking forward: metaplasticity.: The classical Hebbian story says synapses change in response to activity. Bienenstock, Cooper, and Munro [26] proposed

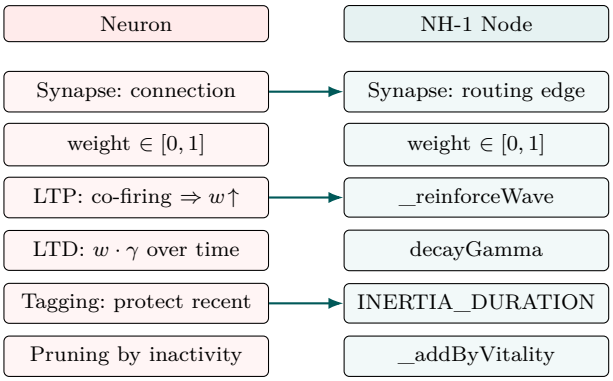

Fig. 1. Direct correspondence between neuromorphic mechanisms (left) and Axona routing-table operations as implemented in NH-1 (right). Every term on the right has one corresponding artifact in the codebase. Arrows are shown on a representative subset to avoid visual clutter; the full mapping is in Table I.

TABLE I
Direct mapping from biological mechanisms to Axona routing-table operations (in the NH-1 implementation).

Neuroscience	Axona (NH-1)
Synapse — a connection between neurons	A peer entry in the routing table (Synapse class)
Synaptic weight — readiness to fire	weight ∈ [0, 1]
LTP — co-firing strengthens the connection	Successful lookup increments weights along the traversed edges (_reinforceWave)
LTD — disuse weakens	Time-based decay each tick (weight *= decayGamma)
NMDA coincidence detection	Weight increments only on successful traffic, not every probe
Synaptic tagging [25]	inertiaDuration epochs prevent eviction of recently reinforced synapses
Pruning of unused connections	_addByVitality evicts the lowest-vitality (weight × recency) edge

that the threshold separating LTP from LTD is itself an adaptive variable, sliding based on the post-synaptic neuron's recent average activity. This is metaplasticity — plasticity of plasticity — and it produces homeostatic self-balancing toward a target activity level rather than runaway potentiation or silence. Axona's parameters are presently fixed constants chosen by sweep. A metaplastic Axona, in which decayGamma, INERTIA_DURATION, and the annealing schedule are themselves observation-driven, is the natural next step (§X).

IV. Axona Architecture (current implementation: NH-1)

Axona is a learning-adaptive DHT layered above an unmodified Kademlia substrate. The contribution sits between Kademlia's k-buckets and the application: every routing decision, every bucket admission, every churn response runs through a single vitality function. This

TABLE II
The twelve rules of Axona (NH-1), organized by operation.

#	Rule	Operation
1	Stratified bootstrap	structure
2	Mixed-capacity (highway%) deployment	structure
3	Action-Potential (AP) routing	navigate
4	Two-hop lookahead	navigate
5	Iterative fallback (= surrogate routing)	navigate
6	Long-Term Potentiation (LTP)	learn
7	Triadic closure	learn
8	Hop caching + lateral spread	learn
9	Incoming promotion	learn
10	Vitality-based eviction	forget
11	Temperature annealing + reheat	explore
12	Epsilon-greedy first hop	explore

section walks the architecture in five operations, presents the unified admission gate that replaces five distinct mechanisms in the predecessor NX-17, and gives end-to-end pseudocode for one lookup. The architecture is described once and embodied in NH-1, the implementation against which the empirical results in §VII were measured.

A. Five operations, one vitality model

Every Axona behavior decomposes into one of five operations:

- NAVIGATE — choose a next hop given a target.
- LEARN — reinforce edges that carried successful traffic.
- FORGET — decay unused edges; evict by vitality.
- EXPLORE — inject controlled randomness to escape local minima.
- STRUCTURE — bootstrap and steady-state topology maintenance.

Twelve rules implement these operations (Table II); twelve parameters tune them (Table III). Every rule is grounded in either the neuromorphic substrate (§III) or the foundational DHT literature (§II); none was added speculatively.

B. The vitality function

Every synapse s carries two scalars: a learned weight $w_s \in [0, 1]$ and a last-reinforcement epoch t_s. The vitality function combines them multiplicatively:

$$\text{vitality}(s) = w_s \times \text{recency}(s) \tag{2}$$

$$\text{recency}(s) = \begin{cases} 1.0 & \text{if } t - t_s < I \\ e^{-(t - t_s - I)/\tau_r} & \text{otherwise} \end{cases} \tag{3}$$

where t is the current epoch (lookup count), I is the INERTIA_DURATION (default 20), and τ_r is the RECENCY_HALF_LIFE (50). The piecewise definition

implements synaptic tagging [25]: a freshly reinforced synapse is locked at full recency for I epochs — it cannot be evicted regardless of vitality competition — after which recency decays exponentially. Without this lock, learning would be self-destructive: the routing table would discard the very edges it just discovered to be useful.

The weight itself is updated on two events. On every successful lookup (LTP, Rule 6), each synapse s on the traversed path gains $w_s \leftarrow \min(1, w_s + \eta)$ for $\eta = 0.05$. On every tick of the decay clock, every synapse loses $w_s \leftarrow \gamma \cdot w_s$ for $\gamma = 0.995$. A synapse with w_s near 1 that is then unused for ~ 100 ticks decays to ~ 0.6; combined with the recency exponential, its vitality drops below the typical eviction threshold within ~ 200 unused lookups. Frequently used edges accumulate weight faster than decay erodes it; unused edges fade.

C. The synaptome

The bounded, vitality-scored set of outgoing edges at a node is the synaptome, capped at 50 entries (a safe cross-browser WebRTC target; Safari measured at ~ 95 peak connections). The synaptome is Axona's analog of Pastry's three-tier routing state [4] (routing table R, leaf set L, neighborhood set M) consolidated into a single tier scored by vitality. The roles persist in latent form:

- High-vitality entries (recently reinforced, locked) play the leaf-set role: well-trafficked nearby peers, dominate terminal-hop routing.
- Mid-vitality entries cover XOR strata in the manner of Pastry's R: ensures every distance band has a candidate.
- Low-vitality entries plus a 2-hop neighborhood sample play the M role: candidates for annealing replacement, kept warm but not routed through.

A reader familiar with Pastry can think of the synaptome as Pastry's three tables collapsed into a single vitality-scored set. The bookkeeping is unified; the structural roles are unchanged.

D. NAVIGATE: Action-Potential routing

Each lookup hop scores synapses by the Action-Potential metric (Rule 3):

$$\mathrm{AP}(s, k) = \mathrm{progress}(s, k) \times w_s \times \tfrac{1}{2}^{(\ell_s/100)} \qquad (4)$$

where $\mathrm{progress}(s, k) = d(\mathrm{self}, k) - d(s.\mathrm{peer}, k)$ is the XOR-distance reduction toward target k, w_s is the learned weight, and ℓ_s is the observed round-trip latency in milliseconds. The latency penalty halves AP every 100 ms, making nearby fast peers preferred over slightly-better-XOR distant ones. AP routing is therefore latency-aware rather than purely distance-aware: it inherits the locality property that defines Pastry and Tapestry, but evaluates it dynamically per lookup from observed RTT rather than statically at insertion.

a) Two-hop lookahead (Rule 4).: When the top-α AP scores are bunched (no decisive first hop), Axona probes the top $\alpha = 2$ first hops for their best second hops and picks the path with the highest combined $\mathrm{AP}(\mathrm{first}) \times \mathrm{AP}(\mathrm{second})$. The lookahead is exactly two hops — never three — so each step is locally optimal across two hops while compute remains bounded at $O(\alpha \cdot |\mathrm{synaptome}|)$ per step.

b) Iterative fallback (Rule 5).: When AP returns no candidate — every neighbor has progress ≤ 0, i.e. would move away from the target — Axona falls back to k-closest-from-synaptome regardless of progress. This rescues lookups that would otherwise stall in dead corridors. The mechanism is surrogate routing in Tapestry's terminology [5]: when the deterministic next-hop cannot be served, route to the closest live identifier. We adopt the canonical term to make the lineage explicit.

E. LEARN: four reinforcement mechanisms

Every successful lookup triggers four updates that together evolve the synaptome toward the traffic actually flowing through the network:

a) LTP (Rule 6).: On lookup completion, every synapse on the successful path gains weight η and an inertia lock of I epochs. This is the Hebbian rule applied to routing: paths that demonstrably worked become preferred for future queries to the same region.

b) Hop caching plus lateral spread (Rule 8).: Every intermediate node along a successful lookup adds the final lookup target to its synaptome — not its next hop, the target. The path becomes shorter on the next lookup to the same destination from that intermediate node. Lateral spread propagates the new edge to the source's geographic neighbors, so nearby peers see the shortcut on their first lookup, not just after they generate one themselves. This is the key mechanism behind Axona's partition-recovery behavior in §VII-E.

c) Triadic closure (Rule 7).: When a node X observes peer A repeatedly routing through it to peer C, X introduces A directly to C. The open triangle $A-X-C$ closes into a direct edge. The name comes from social-network theory: dense triangles emerge in networks shaped by indirect interaction.

d) Incoming promotion (Rule 9).: Axona tracks which peers reach out to the local node. When an incoming peer accumulates a threshold number of successful uses, it is promoted to a real outgoing synapse via _addByVitality. This is passive learning: peers that want to reach me become candidates for routing through.

F. FORGET: the unified admission gate

The single most important consolidation in Axona is the function _addByVitality, which serves as the admission gate for every synapse insertion — bootstrap, LTP, triadic closure, hop caching, incoming promotion, annealing. It replaces five distinct mechanisms in NX-17: stratified

eviction, two-tier highway management, stratum floors, synaptome floors, and adaptive decay.

```
function _addByVitality(node n, synapse s_new):
  if |n.synaptome| < MAX_SIZE then
    add s_new to n.synaptome
    return
  s_victim ← arg min_{s∈n.synaptome, s∉locked} vitality(s)
  if vitality(s_new) > vitality(s_victim) then
    evict s_victim; add s_new
  else
    refuse silently
```

A continuous decay process (Rule 10, $\gamma = 0.995$ per tick) erodes the weight of every synapse uniformly. Synapses that traffic does not refresh lose vitality and become eligible for replacement; well-used ones stay locked. The combination of LTP-driven bumping, time-driven decay, and vitality-gated eviction is the entire admission system. There are no floors, no tiers, no stratum-specific quotas.

G. EXPLORE: temperature and epsilon

A purely greedy AP-scored router would lock onto early-discovered shortcuts and never find better ones. Axona injects two sources of controlled randomness:

a) Temperature annealing (Rule 11).: Each node carries a temperature $T \in [T_{\min}, T_{\text{init}}]$ (defaults 0.05 and 1.0). T decays multiplicatively per lookup by 0.9997. Each lookup, with probability T, the lowest-vitality synapse is replaced by a 2-hop neighborhood sample. High T means aggressive exploration; low T means greedy exploitation. Reheat: when routing discovers a dead peer, T is spiked to $\max(T, 0.5)$. The system explores when topology has changed, exploits when topology is stable.

b) Epsilon-greedy first hop (Rule 12).: With probability $\epsilon = 0.05$, the first hop of a lookup is replaced by a uniform random sample from the synaptome. This is cheap insurance against a particular failure mode: if the AP-best first hop becomes saturated (success disaster), ϵ-greedy guarantees that every hop occasionally explores alternatives.

H. STRUCTURE: bootstrap and deployment

a) Stratified bootstrap (Rule 1).: A new node is seeded with peers covering all XOR strata uniformly. The mechanism is Pastry's locality-preserving join [4] generalized to Kademlia XOR strata: route a join message through an existing nearby peer, fill the synaptome from nodes encountered along the route. Without stratified bootstrap, the cold-start synaptome is dominated by lucky neighbors — local hops form, long hops do not, and the network never reaches global connectivity.

b) Mixed-capacity deployment (Rule 2).: Real peer-to-peer networks are heterogeneous: most peers run in browsers (WebRTC $\sim$ 100-connection ceiling), some on servers (effectively unlimited). Axona tags a configurable highwayPct fraction of nodes as server-class: unlimited inbound, synaptome cap raised from 50 to 256. The remaining nodes are browser-class. All other mechanisms are identical across both classes. §VII-D shows that 15% server-class nodes captures 70% of the available latency improvement over an all-browser baseline.

TABLE III
The twelve parameters of Axona (NH-1) with default values.

Parameter	Default	Operation
MAX_SYNAPTOME	50	structure
INERTIA_DURATION	20	learn
RECENCY_HALF_LIFE	50	forget
decayGamma	0.995	forget
LTP_eta	0.05	learn
T_init	1.0	explore
T_min	0.05	explore
T_cooling	0.9997	explore
T_reheat	0.5	explore
epsilon	0.05	explore
lookaheadAlpha	2	navigate
MAX_HOPS	40	navigate

I. End-to-end lookup

Putting the operations together, one routing tick proceeds as in Figure 2.

Every operation cost is bounded by $O(|\text{synaptome}|)$. At the 50-synapse cap, per-hop compute is $\sim$0.2 ms — small relative to the 10 ms simulated transit cost.

V. Publish/Subscribe via Axonal Trees

A practical DHT must support broadcast: a publisher should be able to deliver a message to all subscribers of a topic without enumerating them. Layered approaches such as SCRIBE on Pastry [11] and Bayeux on Tapestry [27] demonstrate that routed subscribe plus reverse-path-forwarding multicast is the canonical mechanism. Axona integrates the same mechanism directly into the protocol: pub/sub is a built-in primitive, not an application-layer overlay. The result is an axonal tree (the term reflects the biological analog: an axon is the single output projection of a neuron, branching to many downstream targets) whose shape adapts under churn through routed re-subscribe.

a) Topic identity.: Topics are addressed by deterministic identifiers:

$$\text{topicId} = \text{publisher.cellPrefix}_8 \parallel \text{hash}_{56}(\text{event\_name})$$

The publisher's S2 cell prefix anchors the topic root inside the publisher's geographic neighborhood — naturally close to the audience in many real workloads — and the hash component disambiguates topics within a cell. Both publisher and every subscriber derive the same 64-bit identifier offline, with no negotiation. This solves the "random root" problem of K-closest replication schemes that lose publisher/subscriber agreement under churn.

```
on lookup(target):
  hop = 0
  while hop < MAX_HOPS:
    cands = synaptome U incoming_syns
    next  = AP_score(cands, target)        // NAVIGATE
    if next is None:                       // NAVIGATE
      next = iter_fallback(target)
    if hop == 0 and rand() < epsilon:      // EXPLORE
      next = random(synaptome)
    record_transit(prev, next)             // LEARN: triadic
    promote_incoming_if_warranted()        // LEARN: incoming
    hop_cache_destination()                // LEARN: hop caching
    anneal_replace_lowest_vitality()       // EXPLORE
    hop = hop + 1

on success:    reinforce_path()            // LEARN: LTP
periodically:  decay_all_weights()         // FORGET
```

Fig. 2. End-to-end Axona (NH-1) lookup tick. All five operations (navigate, learn, forget, explore, structure) interleave on every routing step; learning is a side-effect of routing rather than a separate maintenance phase.

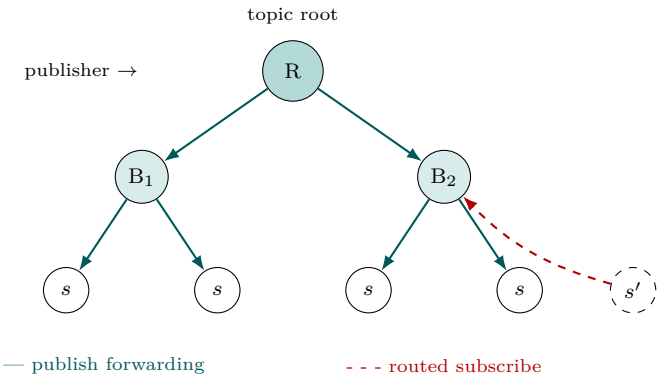

Fig. 3. Axon-tree formation. A new subscriber s' issues a routed subscribe toward topicId (red dashed); the first live axon role on the path (here, B_2) intercepts and adopts it. Publishes flow from the root R through the branches to all subscribers (teal solid). Tree healing under churn is the same mechanism: a subscriber whose parent dies re-issues its subscribe, attaches to whichever live ancestor is now closest, and continues receiving publishes after one refresh interval.

b) Tree formation: routed subscribe.: A subscribe message is routed through the DHT toward topicId. The first node holding an existing axon role for this topic intercepts the subscribe and adds the new subscriber to its children. If the walk completes with no axon found, the terminal node opens a new axon role and becomes the root. This is structurally identical to SCRIBE's reverse-path forwarding tree (Figure 3).

c) Branching: external-peer batch adoption.: When an axon's direct child count exceeds maxDirectSubs, it selects a synaptome peer (external to its current children) as a new sub-axon, partitions its current children by XOR proximity to the sub-axon, and ships the relevant batch in a single adopt_subscribers message. Two invariants prevent runaway recursion: branches always go to peers outside the current subtree, and the new branch's capacity is checked before adoption. Tree depth grows as $O(\log_C S)$ in subscribers S given capacity C.

d) Self-healing: re-subscribe as liveness.: Every axon role re-issues its subscribe on a fixed refresh interval (default 10 s). The walk lands on whichever live axon is closest to topicId now. If the parent died, the re-subscribe attaches to a different live ancestor. If the tree was reorganized, the change is invisible to the subscriber. The re-subscribe is the liveness check — there is no separate ping, no parent tracking, no gossip. The protocol exploits the same mechanism that builds the tree to repair it.

e) Temporal pub/sub.: A subscribe is not just "send me future messages." It is "send me every message I have not already seen." Each subscribe carries a lastSeenTs — the highest publish timestamp the subscriber has observed. Every relay maintains a bounded ring buffer (default 100 messages per topic). On subscribe arrival, the relay replays filtered cache entries with publishTs > lastSeenTs as a single batched message before forwarding the subscribe upstream. Healing and replay are the same mechanism: a subscriber attaching to a new branch after parent death receives both the attachment and any messages that elapsed during the gap.

A. Production refinements

Live deployment of the axonal pub/sub layer on a real WebRTC mesh surfaced three corner cases the simulator's abstract network model had not exercised. Each is fixed in the protocol layer; the production semantics diverge slightly from the simulator-textbook description above.

a) Lazy-axon promotion.: The simulator implementation drops a publish_k that lands at a K-closest node which does not yet hold a role for the topic — falling back to a routed walk that lands on another empty-role node and gives up. In a real network, this manifests as publish-before-subscribe lost messages: the publisher emits content before any subscriber has registered, the K-closest peers silently discard it, and by the time subscribers attach the messages are gone. The fix is to promote any K-closest receiver of an unhandled publish_k to a (childless) root role immediately and seed the replay cache. When a

subscriber later attaches, the existing replay path serves the cache. Empty roles garbage-collect after rootGraceMs (default 60 s) so cost remains bounded: nodes that never accumulate subscribers shed their role on the next refresh sweep.

b) Self-replay (**lastSeenTs** override).: A subtler interaction arises from the dual role of lastSeenTs. When a K-closest node receives a publish_k it advances lastSeenTsByTopic[topicId] to the latest cached publishTs — recording "I have seen on the wire." If that same node later subscribes to the topic (a common case since K-closest is computed from local synaptome state and may include the node itself), the subscribe payload carries that lastSeenTs, and the cache filter publishTs > lastSeenTs excludes every cached message — even though those messages reside in the node's own cache and have never been delivered to a local handler. The application-level view ("messages delivered to my subscription") is logically independent of the network-level view ("publishIds my relay has observed"). For the self-replay case (subscriberId = this.nodeId), the protocol ignores lastSeenTs and dispatches the full cache directly into the local delivery callback.

c) Multi-hop sendDirect (tunneled-direct).: The K-closest replication model assumes any pair of peers can communicate in one hop. In a real WebRTC mesh, this is only true when both endpoints have established a direct DataChannel — something NAT traversal does not always achieve. The simulator implementation of sendDirect is a single transport.notify call; with an unbound target, the call returns successfully but the message is silently dropped. Under sparse mesh conditions, 4 of 5 fan-out attempts evaporate this way, collapsing the K-replicated topic into whichever single peer remains directly bound. The production fix is a transport-layer fallback: if the target is not directly bound, wrap the message in a ___tunneled_direct___ routed envelope and let the existing Axona routing walk hop-by-hop toward the target. A small handler at the destination unwraps the envelope and dispatches into the same direct-handler table that a single-hop delivery would have hit. The bridge becomes a routing hop among many rather than a privileged relay.

d) Stress validation.: A 100-peer in-process stress harness exercises three scenarios in a single Node.js process with no bridge, no WebSockets, and no WebRTC — pure protocol semantics. (1) One publisher, 99 subscribers, three messages: 99/99 full delivery, K=5 axons plus recruited sub-axons. (2) Publish-before-subscribe: publisher emits five messages with no subscribers, then 50 subscribers join late: all 50 receive all five via replay. (3) Multi-topic: five publishers, five topics, 95 subscribers round-robin: every topic delivered to every assigned subscriber, with K=5 axons per topic spread across the network. Per-peer participation profile at $N = 100$: 62 subscriber-only, 33 subscriber+axon, 3 publisher+subscriber, 1 publisher+axon, 1 all-roles. No idle peers, axon-roles distribute as the K-closest set varies per topic.

VI. The Geographic DHT (G-DHT)

The Geographic DHT (G-DHT) is the second protocol contribution of this paper. It is a simple structural extension of Kademlia that gives the overlay an awareness of physical locality without changing the routing algorithm itself. G-DHT serves throughout the rest of the paper as the geographically-aware substrate on which Axona's learning is layered, and as the comparative baseline against which Axona (NH-1) is measured.

A. Construction

Every node identifier is composed by concatenating an S2 cell prefix with a public-key hash:

$$\text{nodeId} = \underbrace{\text{S2cell}_8}_{8 \text{ bits}} \parallel \underbrace{H(\text{publicKey})}_{56 \text{ bits}} \tag{5}$$

The S2 library [28] maps every point on Earth to a 64-bit cell ID along a Hilbert space-filling curve projected through six cube faces. The top 8 bits encode the cube face plus a 5-bit Hilbert subdivision, yielding $6 \times 32 = 192$ continent-scale tiles worldwide (typical examples: "western North America," "South-East Asia"). The Hilbert curve property ensures that geographically adjacent points have numerically close cell IDs: XOR distance in identifier space approximates physical distance, for free.

The remaining 56 bits are the standard cryptographic hash of the node's public key. There is no other change to Kademlia: K-buckets, α-parallel find_node, and the iterative refinement algorithm are all retained verbatim. The locality property emerges from the ID structure alone, with no algorithmic complexity added.

B. Why this matters: the back-of-the-envelope

A pure Kademlia overlay with $N = 10^6$ uniformly-distributed nodes requires $\log_2 N \approx 20$ hops per lookup. Each hop traverses a random pair of nodes separated on average by half the planet, with about 100 ms one-way round-trip time. The naive estimate gives an average message time of $20 \times 100 = 2$ seconds.

G-DHT changes the calculation. Most of a typical user's traffic is geographically local — social, transactional, real-time. With the location prefix, local connections are XOR-close in identifier space, so a regional lookup operates within a much smaller effective network. Suppose a particular region holds $\sim 10\,000$ nodes with typical inter-node RTT of ~ 7 ms within a continental cell. Then $\log_2(10\,000) \approx 13$ hops $\times$ 7 ms ≈ 91 ms for an in-region message — more than a 20$\times$ improvement over the global Kademlia number on the same total network.

The mechanism for cross-region routing is what we call a network of networks: when communicating with a far-away node, G-DHT first locates any peer inside the destination's geographic neighborhood, then refines locally

inside that neighborhood. The long-haul hop happens once per lookup, not at every step. Global lookups also benefit: at 25 000 nodes, G-DHT's measured global latency is 287 ms versus K-DHT's 503 ms (Table IV) — a 1.75× improvement even when no part of the lookup is regional.

C. Geography is a seed, not a teacher

G-DHT is not a learning protocol. The S2 prefix does not teach locality — it structures the identifier space so that XOR-distance routing accidentally tracks physical distance for the dominant traffic patterns. The routing table is fixed at insertion time and lazily repaired thereafter. This is the same class of mechanism as Pastry's locality-preserving join [4] or Tapestry's RTT-sorted routing table [5], and it shares the same limitation: without learning, the table cannot improve as the network changes. Axona's contribution (§IV) is to layer traffic-driven learning on top of G-DHT's structural locality. The geography ablation in §VII-C confirms that the two contributions are independent: with the G-DHT prefix removed (geoBits = 0), Axona still routes 8% faster than Kademlia purely from learning.

D. Cooperative trust and security tradeoff

The S2 prefix is self-declared. A node can claim any prefix it wants; the protocol does not verify the claim against any external ground truth. This is acceptable when locality benefits are cooperative — well-behaved peers do not lie, and an attacker who falsifies a prefix degrades only the locality benefit, not routing correctness. The adversarial consequences are real: a coordinated set of nodes claiming the same prefix can mount a Sybil swarm in a target cell or eclipse a regional namespace. Mitigations (Vivaldi-style learned coordinates, geographic proof-of-work, IP-ASN binding) are detailed in §X. None of these are implemented in the present G-DHT; the tradeoff between locality benefit and identity-region disclosure is, today, a deployment choice. A privacy-sensitive deployment may choose to spoof the prefix, accepting reduced locality in exchange for region anonymity.

VII. Evaluation

This section reports four headline empirical results: NH-1 at the Dabek 3δ analytical floor (§VII-B); the geography-stripped ablation showing learning still wins (§VII-C); heterogeneous-capacity deployment (§VII-D); and pub/sub robustness including the single-bridge Slice World partition test (§VII-E). All data and the simulator that produced it are open-source [12].

A. Methodology

The simulator is ∼25 000 lines of JavaScript. Latency is computed geometrically (great-circle distance scaled to a per-hop one-way propagation delay plus 10 ms processing) rather than waited on, so the simulation runs at full CPU speed. The implementation deliberately abstracts wall-clock transport, encryption, and node identity to focus on routing dynamics; §VIII discusses what this abstraction misses. Five integrity invariants are enforced after every churn event: (i) bilateral connection-cap audit; (ii) locality preservation (no optimization reads another node's routing table directly); (iii) bounded K-element RPC envelopes; (iv) per-protocol parameter routing; (v) geographic-prefix flow into every protocol's node-ID construction.

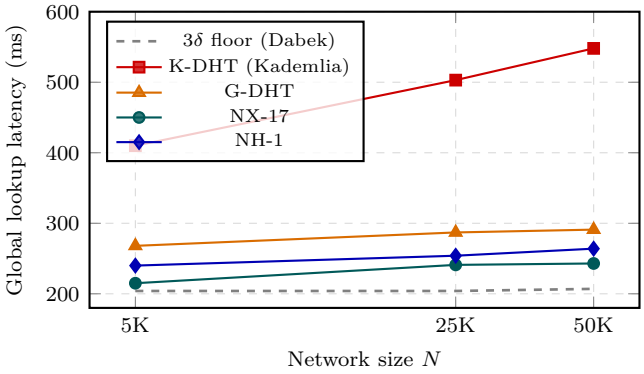

Fig. 4. Global-lookup latency vs network size, plotted against the Dabek 3δ analytical floor (gray dashed). NX-17 plateaus at 1.18× the floor at 25 000 and 50 000 nodes; NH-1 at 1.28×. Kademlia worsens from 2.01× to 2.65× over the same range. The floor itself is essentially flat (δ is a property of the network, not of N).

The headline measurements use 25 000-node networks; we validate at 5 000 and 50 000 nodes for the 3δ result. Default configuration: web-limited 100-connection cap (browser-class), 50 synaptome cap, $K = 20$, $\alpha = 3$, 500 lookups per test cell, omniscient initialization (every protocol seeded with its theoretically optimal K-closest neighbors) to isolate routing-algorithm contribution from bootstrap variance. Where canonical initialization is used — every protocol forced to start from identical K-closest XOR routing tables — we say so explicitly.

B. Headline: NH-1 at the Dabek 3δ floor

The Dabek bound (Eq. 1) requires a measured value of δ, the median pairwise one-way internet latency for the population under test. We compute δ at the start of every benchmark sweep by sampling 10 000 random node pairs and taking the median of messageLatency(a, b) for each. This is independent of any protocol and is therefore a property of the network, not of the DHT. Table IV reports δ together with each protocol's measured global-lookup latency at three network sizes.

The result has three parts. Figure 4 plots the protocol curves against the analytical reference line.

a) NH-1's predecessor plateaus at the floor.: NX-17 sits at 1.18× the 3δ floor at both 25 000 and 50 000 nodes — only 36 ms above the analytical lower bound for any recursive $O(\log N)$ DHT. Within the geometric-series decomposition of Eq. 1, the residual 18% overhead has a clean structural explanation: NX-17 averages 4.5 hops where a Dabek-ideal protocol would take ∼3. Each

TABLE IV
Global-lookup latency (ms) versus the 3δ analytical floor. The parenthesized value is multiplicative overhead over the floor. δ is the median pairwise one-way latency for that population, sampled per sweep over 10 000 random node pairs.

N	δ (ms)	3δ floor (ms)	K-DHT	G-DHT	NX-17	NH-1
5 000	67.9	204	410 (2.01×)	268 (1.32×)	**215** (**1.06**×)	240 (1.18×)
25 000	68.0	204	503 (2.46×)	287 (1.41×)	**241** (**1.18**×)	254 (1.25×)
50 000	68.9	207	548 (2.65×)	291 (1.41×)	**243** (**1.18**×)	264 (1.28×)

TABLE V
Geography ablation. Global-lookup latency in milliseconds at 25 000 nodes, canonical initialization. "Improvement vs. Kademlia" isolates the contribution of learning under no-geography conditions.

Protocol	geoBits=0	geoBits=8	Improvement
K-DHT	506	508	— (baseline)
G-DHT	780[†]	284	— (geo is mechanism)
NX-17	**376**	**241**	**−26**% from learning
NH-1	467	269	−8% from learning

[†]G-DHT's geoBits = 0 result reflects its noProgressLimit = 3 lookup-tuning choice; with no geographic gradient the extra "no progress" rounds are wasted overhead.

TABLE VI
Latency under mixed-capacity deployment. "Highway %" is the fraction of server-class peers (synaptome cap raised from 50 to 256, unlimited inbound). The remaining peers stay browser-class.

Highway %	Global ms	500 km ms	2000 km ms
0 (all browser)	263	96	123
5	248	81	108
15 (knee)	**223**	**69**	**98**
30	223	60	83
50	212	55	84
100 (all server)	206	45	74

"extra" hop costs $\sim\delta/2 \approx 34$ ms — exactly the geometric tail Dabek's series predicts.

b) Kademlia worsens with N.: K-DHT goes from 2.01× at 5 000 to 2.65× at 50 000. The log-N hop tax compounds: more hops, each at full pairwise latency, each contributing fully to the total. Kademlia is locality-blind by construction.

c) δ is uncannily close to Dabek's measurement.: Our simulator's median pairwise one-way latency is 67–69 ms across network sizes. Dabek et al.'s King-dataset measurement reports a median of 67 ms [1]. The simulator's geometric latency model and Dabek's wide-area Internet measurements agree quantitatively — which validates the bound's transferability from the King dataset to our population-distributed simulator.

C. Learning beats locality without geography

A skeptical reader of §VII-B might suspect the geographic prefix is doing all the work. To answer this, we strip the prefix (geoBits = 0, identifiers are random hashes of public keys with no geographic structure) and re-run the comparison with canonical initialization at 25 000 nodes (Table V).

The headline: with no geographic information whatsoever, NX-17 routes lookups 26% faster than Kademlia and NH-1 routes them 8% faster — purely from LTP-driven path reinforcement. Add the S2 prefix back in and the gap widens further. This is the cleanest possible "learning helps" claim: every confound (bootstrap strategy, identifier structure, geographic prefix) is controlled, and the routing/learning algorithm still wins. Geography helps; geography is not necessary.

D. Heterogeneous capacity (highway%)

Real peer-to-peer networks are heterogeneous. We sweep the fraction of server-class nodes from 0 to 100% at 25 000 total nodes; Table VI reports five points.

The result: 15% server-class nodes captures 70% of the available improvement over the all-browser baseline (263 → 223 vs 263 → 206 for 100% server). Beyond 15%, additional server capacity yields diminishing returns. This is a deployment-realistic finding: a real network does not need a uniform server fleet.

E. Pub/sub robustness and partition healing

a) Steady-state delivery.: At 25 000 nodes with 79 topic groups of 32 subscribers each, NH-1 delivers 100.0% of publishes in steady state. Under 5% instantaneous churn, immediate delivery falls to 99.9% and recovers to 100.0% after three refresh intervals. There are no orphans, no dead children, and the publisher-subscriber view overlap (subscribers' top-K vs publisher's top-K) remains at 99.5%.

b) Graceful degradation under sustained churn.: Continuous churn at 1% of alive nodes per 5 ticks, with three refresh passes between kills, produces a smooth degradation curve: 98.7% delivered at 5% cumulative churn, 91.2% at 10%, 86.5% at 20%, 70.0% at 25%, 52.4% at 30%. Crucially, there is no cliff: delivery degrades linearly with churn, indicating that the protocol finds a steady state where new-subscriber recruitment matches loss. The dominant residual failure mode at high churn is not broken routing but subscribers temporarily captured at relays that have lost their delivery path to the root — the failure mode the temporal replay cache (§V) was designed to close.

TABLE VII
Slice World single-bridge partition recovery. "Slice success %" is the lookup success rate over a 500-lookup cross-partition workload after partition setup.

Protocol	Slice success %	Slice ms
K-DHT (Kademlia)	0.0%	— (no recovery)
G-DHT	4.6%	525 (no recovery)
NX-17	94.8%	423
NH-1	**94.4**%	**462**

c) Slice World: partition recovery from a single bridge.: The Slice World test partitions the network into Eastern and Western hemispheres connected through a single bridge node placed near Hawaii. Every cross-hemisphere edge except those incident on the bridge is removed before the test begins. The question is not "can the protocol find the bridge?" — that is a one-shot routing problem. The question is: given the single bridge, can the protocol leverage repeated successful crossings to dissolve the partition?

Table VII reports the result. K-DHT achieves 0% success: with no learning, the partition is permanent. G-DHT achieves 4.6%: the geographic prefix happens to point a few peers at the bridge, but no mechanism amplifies the discovery. NX-17 and NH-1 both achieve $\sim$94% success.

The mechanism is the bridge as seed crystal. A diagnostic run at 5 000 nodes shows the recovery directly. After a fresh partition (0 cross-hemisphere synapses), running just 10 lookups yields 20 new cross-hemisphere synapses (an average of $\sim$3 per successful crossing). Hop caching (Rule 8) installs the destination in every intermediate node along the bridge path. Triadic closure (Rule 7) introduces direct edges between observed transit pairs. Lateral spread propagates the new edge to the source's geographic neighbors. By 500 lookups, hundreds of cross-hemisphere synapses exist; the partition has effectively dissolved. The 94% aggregate success rate is the integral of recovery, not a snapshot of bridge-finding. NH-1 and NX-17 route through the partition not by finding the bridge each time — they dissolve the partition.

F. Bandwidth concentration: empirical resolution

The companion red-team analysis [13] ranks bandwidth saturation and congestion collapse as a critical deploy blocker (Issue #3). Of the four Tier 1 issues, three (connection setup, RPC timeouts, latency jitter) are textbook engineering problems with known mitigations. Issue #3 was different: the simulator's uniform-cost-per-hop model (§VII-A) made it architecturally unfalsifiable whether NH-1's adaptive routing concentrated load (the success-disaster oscillation hypothesized by the red team) or distributed it. Until that question was answered, every other claim about the protocol was contingent.

a) Methodology.: We instrument every routing chokepoint with per-node send/receive counters: the SimulatedNetwork.send path used by Kademlia and G-DHT, plus a parallel _msg(from, to, type) helper invoked from each conceptual wire site in NH-1 and the NX-6 lineage (NX-6 through NX-17) where routing bypasses the simulated transport via direct map access. Sites include: each lookup-hop transition; two-hop AP probes; hop-caching writes; lateral-spread propagation; LTP reinforcement walks; triadic-closure introductions; bilateral bootstrap handshakes; the 2-hop neighbour scan used by _localCandidate (annealing and dead-peer replacement); pub/sub routeMessage forwarding; and sendDirect entry. Per-cycle counters are reset after each training session, so the captured numbers are deltas.

For each cycle we report total messages, mean / p50 / p99 / max per-node loads, the Gini coefficient of the per-node distribution, and counts of nodes exceeding 10× and 100× the cycle mean. The 100× threshold corresponds to nodes whose absolute forwarding rate is two orders of magnitude above peers — the practical definition of a bandwidth-saturation hazard.

b) Headline result: distribution shape, 4×4 sweep.: Across 42 runs spanning four protocols (K-DHT, G-DHT, NX-17, NH-1) and four sizes (5K, 10K, 25K, 50K), the load distribution diverges sharply between classical and adaptive protocols (Table VIII).

The hypothesis the red team raised — that adaptive routing concentrates more than static routing — is empirically inverted. Kademlia and G-DHT develop 56–62 catastrophic-bandwidth nodes at 50K (max/mean ratios $> 200\times$); NH-1 produces zero such nodes at any tested scale. The classical DHTs' Gini coefficient grows monotonically with N ($0.876 \to 0.940$ from 5K to 50K), while NH-1's grows much more slowly ($0.549 \to 0.661$).

c) Long-run stability.: A 50-session run at 50K nodes confirms the steady state is genuinely steady. Gini holds in $[0.657, 0.668]$ across all 50 cycles (standard deviation 0.0025); first-half versus second-half drift is +0.002. Hot $>$ 10× count holds in $[1378, 1453]$; hot $>$ 100× count is zero throughout. Average hops trend gently downward ($5.55 \to 5.27$): the network keeps learning without disturbing the load profile.

d) Decomposition of the 25× volume tax.: NH-1's total traffic is 25–30× Kademlia's. Per-type instrumentation isolates the cost. At 50K:

A single mechanism — the 2-hop neighbour scan in _localCandidate, used by simulated annealing and dead-peer replacement — accounts for 89% of NH-1's wire traffic. We introduce a parameter annealRateScale that multiplies the per-hop annealing trigger probability without disturbing the temperature-reheat semantics that fire on dead-peer detection (preserving churn-recovery behaviour).

e) The rate-knee.: Sweeping annealRateScale $\in \{0.05, 0.10, 0.25, 0.50, 1.00\}$ at 50K nodes (Table X), local_probe scales linearly with the rate (perfect 10×

TABLE VIII
Steady-state per-cycle traffic distribution at 50 000 nodes, 500 lookups per cycle, 10 training sessions. "Hot > 100×" counts nodes processing more than 100× the network mean per cycle — the bandwidth-saturation hazard threshold.

Protocol	Total/cyc	Gini	max/mean	Hot > 10×	Hot > 100×
K-DHT	14,463	**0.940**	208×	551	**56**
G-DHT	17,633	**0.932**	213×	423	**62**
NX-17	359,459	0.694	61×	1,524	**0**
NH-1	416,071	0.661	47×	1,419	**0**

TABLE IX
Per-type traffic decomposition for NH-1 at 50 000 nodes, 500 lookups per cycle. Counts are total sender-plus-receiver bumps; wire-message count is half. Routing-hop find_node traffic is 1.3% of all traffic; the remaining 98.7% is learning side-effects.

Message type	Count/cyc	%
local_probe (2-hop annealing scan)	370,566	**89.1**%
lookahead_probe (two-hop AP)	20,525	4.9%
lateral_spread	7,474	1.8%
hop_cache_lateral	6,404	1.5%
find_node (actual routing hop)	5,538	1.3%
hop_cache	3,737	0.9%
reinforce (LTP feedback)	1,823	0.4%

reduction at rate 0.10). Routing quality is unchanged across the entire range: hops 5.48–5.54, time 268–271 ms, success 100%. The knee is at rate ≈ 0.10 — the setting at which absolute hot-node count drops to its minimum (575 at rate 0.10 versus 1,428 at rate 1.00 and 852 at rate 0.05).

f) Robustness under churn.: Under 5% per-cycle node turnover at 25 000 nodes (Table XI), throttled NH-1 holds 100% routing success with Gini 0.79 — still strictly better than Kademlia's 0.92 — and zero hot > 100× nodes. Both classical protocols develop 7 catastrophic-bandwidth nodes under the same churn load. The T_REHEAT mechanism still fires on dead-peer detection regardless of the base annealing rate, so churn-recovery is uncompromised by the throttle: NH-1's churn-induced Gini increase (+0.012) is the smallest of any protocol tested.

g) Reclassification of red-team Issue #3.: The foundational concentration question is empirically resolved: NH-1 distributes load broadly and stably; classical DHTs concentrate it catastrophically at scale. The 25× volume tax of full annealing is a single tunable parameter with a clean knee and zero impact on routing quality. The remaining bandwidth-saturation concern — a popular pub/sub topic's root saturating from publish volume under Zipf workloads — is a bounded sub-problem whose mitigation (hot-axon-root migration plus MaxDisjoint replication, §X) was already specified by the red team. The deploy-blocker framing of the broader issue is retired.

VIII. Discussion and Limitations

A. What Axona adds beyond predecessors

a) Versus Pastry and Tapestry.: Both put locality in the routing table; both populate routing-table slots with closest-by-RTT peers at insertion time. Axona retains this property dynamically through AP scoring and extends it: routing tables evolve via LTP, hop caching, triadic closure, and lateral spread, rather than being set once at insertion and lazily repaired.

b) Versus Coral DSHT.: Coral builds nested RTT clusters explicitly. Axona consolidates the cluster hierarchy into a single vitality-scored set: the high-vitality core plays the local-cluster role, mid-vitality entries cover wider strata, and low-vitality peers serve as candidate replacements during annealing. The structural choice is opposite (one flat table vs. multiple nested) but the goal — latency-aware routing — is shared.

c) Versus SCRIBE on Pastry.: Axona integrates pub/sub directly into the protocol rather than layering it on top. The mechanism (routed subscribe + reverse-path forwarding) is identical to SCRIBE; the difference is that Axona's axon trees adapt under churn through routed re-subscribe alone, with no replication, no gossip, and no parent tracking. This is structurally simpler and (per §VII-E) measurably effective.

B. Measurement limitations

a) Warmup dependency.: Axona needs ∼ 5 000 lookups before the synaptome converges to its trained asymptote. The first minute of deployment is suboptimal. We address this in §VII-A via warmup, but a freshly bootstrapped real network would not have this luxury.

b) Synaptome cap of 50.: A deliberate cross-browser WebRTC target (Safari measured at ∼ 95, Firefox at ∼ 190, Node.js at ∼ 200). Server deployments could reasonably use 200–500. The cap directly constrains the asymptote of LTP-driven specialization and is therefore an architectural limit, not a tuning knob.

c) Locality is structural, not learned.: Without the S2 prefix, the AP scoring's $\frac{1}{2}^{(\ell/100)}$ latency penalty is in principle sufficient to discover locality from scratch, but our warmup budget is too small to do so reliably (§VII-C shows the geoBits = 0 regional latency degrading by 360%). Future work (§X) replaces the structural prefix with a Vivaldi-style learned coordinate.

TABLE X
annealRateScale knee for NH-1 at 50 000 nodes. Routing quality (hops, time, success) is unchanged across the entire range.

rate	Total/cyc	local_probe	Gini	Hot > 10×	vs default
0.05	63,847	18,224	0.866	852	6.5× cut
0.10	**82,650**	**36,989**	**0.834**	**575**	**5.0**× cut
0.25	137,363	92,453	0.770	735	3.0× cut
0.50	232,749	187,366	0.713	1,215	1.8× cut
1.00	414,929	369,552	0.661	1,428	(default)

TABLE XI
Steady-state distribution at 25 000 nodes with 5% per-cycle churn.

Protocol	Total/cyc	Gini	Hot > 100×	Success
K-DHT	12,725	0.919	**7**	100%
G-DHT	15,817	0.910	**7**	99.9%
NX-17 (default)	496,256	0.678	0	100%
NH-1 (default)	356,358	0.654	0	100%
NH-1 (throttled)	**71,695**	**0.786**	**0**	**100**%

d) Methodology gaps.: The evaluation uses uniform-rate lookups on uniformly distributed targets, and uniform churn. Real workloads are Zipf in publish rate (a few hot topics dominate) and burst-y in churn (peak-hour patterns). Our simulator does not yet stress these.

C. Red-team summary: what the simulator does not model

A companion red-team analysis [13] catalogues thirteen deployment-relevant gaps, ranked by deploy-blocker status, severity, and time-to-fix. We summarize the four critical items because the properties claimed in §VII cannot be verified in production until they are addressed:

a) Connection-setup friction.: The simulator treats new synapses as immediately usable. Real WebRTC requires ICE + STUN/TURN + DTLS, taking 1.5–3 s per new synapse under typical NAT conditions. Axona's most important behavioral results (Slice World re-stitching, annealing replacement, hop-caching cascade) all depend on synapses becoming usable rapidly. Production recovery times will be substantially worse than simulator measurements suggest until CONNECTION_SETUP_MS is modeled.

b) RPC timeouts and asynchronous black holes.: The simulator detects dead nodes instantaneously. Real RPCs to silently-dropped nodes stall for multi-second timeout windows during which messages are lost. The 100% pub/sub recovery under 5% churn measurement assumes instant detection; a production-realistic figure would model RPC_TIMEOUT_MS and asymmetric reply paths.

c) Bandwidth saturation.: Reclassified following §VII-F. The hypothesis that AP scoring concentrates load on overloaded nodes is empirically falsified at the scales we measure: NH-1 produces zero > 100×-mean nodes at any tested size while Kademlia produces 56 at 50K (Table VIII). The residual concern is the Zipf-distributed pub/sub case — a single popular topic's root saturating from publish volume rather than from routing traffic. This is bounded sub-problem; mitigation by hot-axon-root migration (Makris et al. [20] DFE pattern) plus MaxDisjoint topic replication (Harvesf-Blough [19]) is sketched in §X.

d) Latency jitter.: Real RTTs fluctuate ±30% from bufferbloat, asymmetric paths, and queuing. Our distance-derived latency is monotone and clean. The LTP exponential moving average has an easy job in the simulator; high-frequency noise could prematurely decay good synapses or promote lucky-but-unstable ones.

e) Bottom line.: The brain is production-ready. The body is not — but the body is now a measurable, scopeable problem rather than a tangle of interacting mechanisms. §X sketches the path forward; the open simulator becomes the lab bench for the next iteration.

IX. Implementation Architecture: A Two-Layer API

A working DHT has two interfaces, not one. The application above wants to publish, subscribe, and look things up; the network below wants to open a channel, send a message, and notice when a peer goes silent. The protocol is the thing in between. Axona is structured so the protocol code is genuinely between those two layers — neither the application nor the network is hardwired into the routing logic — which is what makes the same body of code reusable in both the simulator and the deployed system.

A. The DHT contract — application-facing

The application sees one running node. The contract is small: eight verbs covering lifecycle, operations, and observability. start / stop bring the node up and down; join(sponsor) / leave add to and depart from the network.

lookup(targetKey) walks the routing layer toward a key; subscribe / unsubscribe / publish ride on the axonal-tree primitive. getNodeId, getSynaptome, getMetrics, and onEvent are the read-only observability surface.

The forbidden methods are as instructive as the present ones. There is no method that enumerates "all nodes," no method that takes a peer-id and returns that peer's state, no method that lets the application mutate the routing table. A DHT instance is responsible for one node's view of the network; the simulator's Engine creates many DHT instances and orchestrates them, but that orchestration is a simulator concern — it does not appear in the contract.

B. The Transport contract — network-facing

One Transport instance per running node; the protocol calls into it. Twelve methods in four bands:

- Lifecycle (start, stop, getLocalNodeId).
- Channel pool (openConnection, closeConnection, isConnected). This maps directly onto synaptome semantics: admit-to-synaptome corresponds to openConnection; evict corresponds to closeConnection. The bilateral connection cap is enforced inside the transport — openConnection resolves with false if the remote refused.
- Messaging (send, notify, onRequest, onNotification). Two outbound primitives: request/response and fire-and-forget. The split matters because production transports pay round-trip latency for send but not for notify. Axona uses send for the routing-tick chain (lookup_step, lookahead_probe, find_closest_set, local_probe) and notify for the LEARN side-effects (reinforce, hop_cache, lateral_spread, triadic_introduce).
- Liveness (onPeerDied, getLatency). The transport runs a 1 Hz ping/pong heartbeat on every open channel; missed pongs trigger onPeerDied. The protocol consumes getLatency for AP scoring and registers an onPeerDied callback that populates a per-node _deadPeers set — the same candidate-filter mechanism a production deployment uses.

The Transport contract forbids the protocol from reaching around it: no nodeMap.get(peerId), no peer.synaptome.values() reads, no cross-peer field access. Every cross-peer read happens via send or notify.

C. The simulator as deployment vehicle

The simulator's SimulatedNetwork and the production WebRTCTransport both implement the same Transport contract. Twelve methods, one signature each. The simulator runs handlers synchronously inside the call (in-process, no real RTT); production runs them through WebRTC data channels with real RTT. The protocol code does not switch on which transport it is using.

This is the property that lets the simulator be the deployment vehicle. Years of Axona benchmark numbers — the hop counts, latency distributions, pub/sub coverage, churn-resilience curves in §VII — transfer directly to production because the same code path runs in both worlds. A 25 K-node parity-gate benchmark (post-refactor v0.70.22 against the pre-refactor v0.70.04 reference) confirmed every protocol within a 10% target band: NH-1 within 5–8% on hops, 1–4% on latency; Kademlia, G-DHT, NX-17 within 0.5–3%. The protocol's central algorithm — the recursive-forwarding lookup_step chain — is free of cross-peer state reads; the remaining nodeMap.get(peerId) sites are sim-only orchestration (node creation/destruction bookkeeping, the simulator's god's-eye stratified bootstrap, and the local self-resolution in lookup). Production replaces the bootstrap path with a BootstrapService contract that handles signaling, manifest signature verification, and QR-code pairing out-of-band.

a) Live deployment.: As of v0.3.53 the production Transport and BootstrapService implementations are live. The browser-resident Axona peer (axona-peer) runs at https://axona.net on Mac, Windows, Linux, iOS, and Android behind real-world NATs. The signaling broker (axona-bridge) at https://bridge.axona.net handles the WebRTC offer/answer exchange for browser-to-browser channels; it carries no application traffic and any operator can stand one up, so the rendezvous role is interchangeable. The product name for the protocol and the running network — peer, bridge, SDK — is Axona; NH-1 is the current Axona implementation, and the code path it runs internally is the same one against which the empirical results in §VII were measured. The first end-user application on the live network is civildefense.io, a tap-to-report incident map whose primitives (signed posts, geographic locality, 24-hour expiry, anonymous P2P) inherit directly from this protocol layer.

X. Future Work

The future-work agenda follows the red-team prioritization [13] and integrates the comparative-literature mechanisms identified in §II.

a) Friction-realistic simulator.: Implement CONNECTION_SETUP_MS with new synapses in a pending state excluded from AP scoring until setup elapses; RPC_TIMEOUT_MS for silently-dropped peers with request/reply path tracing; load-dependent AP scoring; and Gaussian jitter injection on every hop. The resulting measurements would be production-realistic versions of the results in §VII.

b) Vivaldi-style learned locality.: Replace the self-declared S2 prefix with synthetic Euclidean coordinates [8] that converge from observed RTTs. Learned coordinates close the cell-eclipse gap (§VI) by removing the cooperative-trust assumption: an attacker cannot falsify a coordinate that is verified against measured RTT.

c) Byzantine pub/sub via MaxDisjoint replication.: Replicate each axon-tree root at d disjoint locations using Harvesf-Blough's placement formula [19]. With $d = 8$

replicas this delivers 90% pub/sub success at 50% malicious nodes per their measurement. Combined with surrogate routing on the multicast tree, this closes the Byzantine-resistance gap without abandoning the axonal-tree primitive.

d) Adaptive parameter tuning.: Following Ghinita and Teo [17], each peer maintains a rolling estimate of local failure rate μ and join rate λ. Annealing cooling rate, reheat amount, and the staleness threshold for synapse eviction all become functions of (μ, λ) rather than fixed constants. A peer in a stable region anneals slowly; a peer in a high-churn region anneals aggressively, automatically.

e) Metaplastic Axona.: The natural endpoint of the previous direction: expose a single QoS knob (target lookup failure rate) and let the system self-tune underlying constants (decayGamma, INERTIA_DURATION, the annealing schedule, even synaptome capacity) to hit it. This is precisely the metaplasticity of §III's forward-pointer: plasticity of plasticity, with the system maintaining a homeostatic operating point rather than running on hand-picked constants.

XI. Conclusion

We have presented Axona, a learning-adaptive distributed hash table and axonal publish/subscribe protocol whose current implementation, NH-1, operates within 1.18–1.28× the Dabek 3δ analytical floor for recursive $O(\log N)$ routing on a 50 000-node network. Axona consolidates eighteen specialized rules and forty-four parameters from its predecessor NX-17 into twelve rules and twelve parameters governed by a single vitality function — weight × recency — that is a direct application of Hebbian long-term potentiation to overlay routing. Built-in axonal pub/sub achieves 100% baseline and 100% recovered delivery under 5% churn through routed re-subscribe alone. Single-bridge network partitions dissolve through cooperative re-stitching driven by hop caching and triadic closure. Stripping the geographic prefix shows that learning alone outperforms Kademlia by 26%, demonstrating that the contribution is structural rather than a consequence of locality engineering.

The Berkeley/Cambridge DHT lineage — Kademlia plus Pastry plus Tapestry plus SCRIBE — gave us the substrate; vitality-driven learning is the contribution. The production stack — axona-peer (the browser-resident node) and axona-bridge (an interchangeable signaling broker) — runs the same protocol code as the simulator that produced the numbers in §VII. Code, data, simulator, and the companion red-team analysis are open at github.com/axona-net/dht-sim [12]; the live peer and bridge sources are at github.com/axona-net/axona-peer and github.com/axona-net/axona-bridge.

References

[1] F. Dabek, J. Li, E. Sit, J. Robertson, M. F. Kaashoek, and R. Morris, "Designing a DHT for low latency and high throughput," in Proc. USENIX Symposium on Networked Systems Design and Implementation (NSDI), 2004.

[2] I. Stoica, R. Morris, D. Karger, M. F. Kaashoek, and H. Balakrishnan, "Chord: A scalable peer-to-peer lookup service for internet applications," in Proc. ACM SIGCOMM, 2001, pp. 149–160.

[3] P. Maymounkov and D. Mazières, "Kademlia: A peer-to-peer information system based on the XOR metric," in Proc. International Workshop on Peer-to-Peer Systems (IPTPS), 2002.

[4] A. Rowstron and P. Druschel, "Pastry: Scalable, decentralized object location and routing for large-scale peer-to-peer systems," in Proc. IFIP/ACM International Conference on Distributed Systems Platforms (Middleware), 2001, pp. 329–350.

[5] B. Y. Zhao, L. Huang, J. Stribling, S. C. Rhea, A. D. Joseph, and J. D. Kubiatowicz, "Tapestry: A resilient global-scale overlay for service deployment," IEEE Journal on Selected Areas in Communications (JSAC), vol. 22, no. 1, pp. 41–53, 2004.

[6] S. Ratnasamy, P. Francis, M. Handley, R. Karp, and S. Shenker, "A scalable content-addressable network," in Proc. ACM SIGCOMM, 2001, pp. 161–172.

[7] M. J. Freedman, E. Freudenthal, and D. Mazières, "Democratizing content publication with Coral," in Proc. USENIX Symposium on Networked Systems Design and Implementation (NSDI), 2004.

[8] F. Dabek, R. Cox, F. Kaashoek, and R. Morris, "Vivaldi: A decentralized network coordinate system," in Proc. ACM SIGCOMM, 2004.

[9] D. O. Hebb, The Organization of Behavior: A Neuropsychological Theory. Wiley, 1949.

[10] T. V. P. Bliss and T. Lømo, "Long-lasting potentiation of synaptic transmission in the dentate area of the anaesthetized rabbit following stimulation of the perforant path," The Journal of Physiology, vol. 232, no. 2, pp. 331–356, 1973.

[11] M. Castro, P. Druschel, A.-M. Kermarrec, and A. Rowstron, "SCRIBE: A large-scale and decentralized application-level multicast infrastructure," IEEE Journal on Selected Areas in Communications (JSAC), vol. 20, no. 8, pp. 1489–1499, 2002.

[12] D. A. Smith, "N-DHT simulator," https://github.com/YZ-social/dht-sim, 2026.

[13] ——, "NH-1 red team analysis: Critical issues facing deployment," YZ.social, Tech. Rep. v0.3.38, 2026, available at https://github.com/YZ-social/dht-sim/blob/main/documents/NH1-RedTeam-v0.3.38.md.

[14] S. Saroiu, P. K. Gummadi, and S. D. Gribble, "A measurement study of peer-to-peer file sharing systems," Multimedia Systems, vol. 9, no. 2, pp. 170–184, 2003.

[15] R. Mahajan, M. Castro, and A. Rowstron, "Controlling the cost of reliability in peer-to-peer overlays," in Proc. International Workshop on Peer-to-Peer Systems (IPTPS), 2003.

[16] S. Krishnamurthy, S. El-Ansary, E. Aurell, and S. Haridi, "A statistical theory of Chord under churn," in Proc. International Workshop on Peer-to-Peer Systems (IPTPS), 2005.

[17] G. Ghinita and Y. M. Teo, "An adaptive stabilization framework for distributed hash tables," in Proc. IEEE International Parallel and Distributed Processing Symposium (IPDPS), 2006.

[18] M. Castro, P. Druschel, A. Ganesh, A. Rowstron, and D. S. Wallach, "Secure routing for structured peer-to-peer overlay networks," in Proc. USENIX Symposium on Operating Systems Design and Implementation (OSDI), 2002, pp. 299–314.

[19] C. Harvesf and D. M. Blough, "The design and evaluation of techniques for route diversity in distributed hash tables," in Proc. IEEE International Conference on Peer-to-Peer Computing (P2P), 2007.

[20] A. Makris, K. Tserpes, and D. Anagnostopoulos, "A novel object placement protocol for minimizing the average response time of get operations in distributed key-value stores," in Proc. IEEE International Conference on Big Data (BIGDATA), 2017, pp. 3196–3205.

[21] D. Karger, E. Lehman, T. Leighton, R. Panigrahy, M. Levine, and D. Lewin, "Consistent hashing and random trees: Distributed caching protocols for relieving hot spots on the World Wide Web," in Proc. ACM Symposium on Theory of Computing (STOC), 1997, pp. 654–663.

[22] S. B. Laughlin and T. J. Sejnowski, "Communication in neuronal networks," Science, vol. 301, no. 5641, pp. 1870–1874, 2003.
[23] M. F. Bear, B. W. Connors, and M. A. Paradiso, Neuroscience: Exploring the Brain, 4th ed. Jones & Bartlett Learning, 2020.
[24] P. K. Stanton and T. J. Sejnowski, "Associative long-term depression in the hippocampus induced by hebbian covariance," Nature, vol. 339, no. 6221, pp. 215–218, 1989.
[25] U. Frey and R. G. M. Morris, "Synaptic tagging and the late phase of long-term potentiation," Nature, vol. 385, no. 6616, pp. 533–536, 1997.
[26] E. L. Bienenstock, L. N. Cooper, and P. W. Munro, "Theory for the development of neuron selectivity: Orientation specificity and binocular interaction in visual cortex," Journal of Neuroscience, vol. 2, no. 1, pp. 32–48, 1982.
[27] S. Q. Zhuang, B. Y. Zhao, A. D. Joseph, R. H. Katz, and J. D. Kubiatowicz, "Bayeux: An architecture for scalable and fault-tolerant wide-area data dissemination," in Proc. International Workshop on Network and Operating System Support for Digital Audio and Video (NOSSDAV), 2001, pp. 11–20.
[28] Google, "S2 geometry library," https://s2geometry.io/, 2011.